

EC200A-CN(TA) QuecOpen SIM API 参考手册

LTE Standard 模块系列

版本：1.0.0

日期：2022-10-13

状态：临时文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登陆网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2022，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2022.

文档历史

修订记录

版本	日期	作者	变更表述
-	2022-06-20	Lyndsay XIE	文档创建
1.0.0	2022-10-13	Lyndsay XIE	临时版本

目录

文档历史	3
目录	4
表格索引	6
1 引言	7
2 SIM API	8
2.1. 头文件	8
2.2. 函数概览	8
2.3. 函数详解	9
2.3.1. ql_sim_init	9
2.3.2. ql_sim_deinit	10
2.3.3. ql_sim_get_imsi	10
2.3.3.1. QL_SIM_SLOT_E	11
2.3.3.2. QL_SIM_APP_TYPE_E	11
2.3.4. ql_sim_get_iccid	12
2.3.5. ql_sim_get_phone_num	12
2.3.6. ql_sim_get_operators	13
2.3.6.1. ql_sim_operator_list_t	13
2.3.6.2. ql_sim_operator_t	14
2.3.7. ql_sim_enable_pin	14
2.3.7.1. QL_SIM_PIN_E	15
2.3.8. ql_sim_disable_pin	16
2.3.9. ql_sim_verify_pin	16
2.3.10. ql_sim_change_pin	17
2.3.11. ql_sim_unblock_pin	18
2.3.12. ql_sim_get_card_info	18
2.3.12.1. ql_sim_card_info_t	19
2.3.12.2. QL_SIM_CARD_STATE_E	19
2.3.12.3. QL_SIM_CARD_TYPE_E	20
2.3.12.4. ql_sim_app_info_t	21
2.3.12.5. QL_SIM_APP_STATE_E	21
2.3.13. ql_sim_read_file	22
2.3.13.1. ql_sim_file_t	23
2.3.14. ql_sim_write_file	23
2.3.15. ql_sim_get_file_info	24
2.3.15.1. ql_sim_file_info_t	25
2.3.15.2. QL_SIM_FILE_TYPE_E	25
2.3.16. ql_sim_read_phone_book	26
2.3.16.1. ql_sim_phone_book_record_t	27
2.3.17. ql_sim_write_phone_book	27
2.3.18. ql_sim_open_logical_channel	28
2.3.19. ql_sim_close_logical_channel	28

2.3.20.	ql_sim_send_apdu	29
2.3.20.1.	ql_sim_apdu	29
2.3.21.	ql_sim_set_card_status_cb	30
2.3.21.1.	ql_sim_card_status_cb_f	30
2.3.22.	ql_sim_set_service_error_cb	31
2.3.22.1.	ql_sim_service_error_cb_f	31
3	示例	32
3.1.	获取 SIM 卡的 IMSI	32
3.2.	获取 SIM 卡的 ICCID	33
3.3.	获取 SIM 卡的电话号码	34
3.4.	获取运营商列表	36
3.5.	PIN 码操作	37
3.5.1.	使能 PIN 码	37
3.5.2.	禁用 PIN 码	40
3.5.3.	验证 PIN 码	40
3.5.4.	更改 PIN 码	40
3.5.5.	解锁 SIM 卡并设置新 PIN 码	43
3.6.	SIM 卡文件操作	45
3.6.1.	从 SIM 卡文件读取数据	45
3.6.2.	把数据写入 SIM 卡文件	48
3.6.3.	获取 SIM 卡文件信息	50
3.7.	获取 SIM 卡状态	52
3.8.	电话簿操作	54
3.8.1.	读取电话簿信息	54
3.8.2.	修改电话簿信息	55
3.9.	发送 APDU	58
3.9.1.	打开逻辑通道	58
3.9.2.	关闭逻辑通道	59
3.9.3.	发送 APDU	60
4	附录 参考文档及术语缩写	62

表格索引

表 1: 函数概览	8
表 2: 参考文档	62
表 3: 术语缩写	62

Preliminary Document
Not Checked

1 引言

移远通信 EC200A-CN(TA)模块支持 QuecOpen®方案。QuecOpen®是基于 Linux 的嵌入式开发平台，可简化 IoT 应用的软件设计和开发过程。有关 QuecOpen®的详细信息，请参考文档 [1]。

SIM 卡为用户身份识别卡、智能卡，GSM 数字移动电话机必须装上此卡方能使用。SIM 卡芯片上存储了数字移动电话客户的信息、加密的密钥以及用户的电话簿等内容，可供 GSM 网络客户进行身份鉴别，并对客户通话时的语音信息进行加密。本文档所介绍的 SIM 卡泛指 ICC 和 UICC 卡。

本文档详细描述了在 QuecOpen®方案下，移远通信 EC200A-CN(TA)模块的 SDK 中提供的 SIM API。

2 SIM API

2.1. 头文件

SIM API 的头文件为 `ql_sim.h`，位于 SDK 包的 `ql-sysroots\usr\include\ql-sdk\` 目录下。若无特别说明，本文档所涉及头文件均在该目录下。

2.2. 函数概览

表 1：函数概览

函数	说明
<code>ql_sim_init()</code>	初始化 SIM 卡服务
<code>ql_sim_deinit()</code>	注销 SIM 卡服务
<code>ql_sim_get_imsi()</code>	获取 SIM 卡的 IMSI
<code>ql_sim_get_iccid()</code>	获取 SIM 卡的 ICCID
<code>ql_sim_get_phone_num()</code>	获取 SIM 卡的电话号码
<code>ql_sim_get_operators()</code>	获取运营商列表
<code>ql_sim_enable_pin()</code>	使能 PIN 码
<code>ql_sim_disable_pin()</code>	禁用 PIN 码
<code>ql_sim_verify_pin()</code>	验证 PIN 码
<code>ql_sim_change_pin()</code>	更改 PIN 码
<code>ql_sim_unblock_pin()</code>	解锁 SIM 卡并设置新 PIN 码
<code>ql_sim_get_card_info()</code>	获取 SIM 卡信息
<code>ql_sim_read_file()</code>	从 SIM 卡文件读取数据

<code>ql_sim_write_file()</code>	把数据写入 SIM 卡文件
<code>ql_sim_get_file_info()</code>	获取 SIM 卡文件信息
<code>ql_sim_read_phone_book()</code>	读取 SIM 卡电话簿中保存的联系人信息
<code>ql_sim_write_phone_book()</code>	修改电话簿内保存的联系人信息
<code>ql_sim_open_logical_channel()</code>	打开逻辑通道
<code>ql_sim_close_logical_channel()</code>	关闭逻辑通道
<code>ql_sim_send_apdu()</code>	发送 APDU
<code>ql_sim_set_card_status_cb()</code>	设置 SIM 卡状态接收回调函数
<code>ql_sim_set_service_error_cb()</code>	设置 SIM 卡服务异常接收回调函数

备注

若无特别说明，本文档所述函数均不支持并发调用，并且不能在相关回调函数中调用以上函数。

2.3. 函数详解

2.3.1. ql_sim_init

该函数用于初始化 SIM 卡服务。

● 函数原型

```
int ql_sim_init(void)
```

● 参数

无

● 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 `ql_type.h`。

备注

使用其他 SIM API 前，必须调用本函数初始化 SIM 卡服务。

2.3.2. ql_sim_deinit

该函数用于注销 SIM 卡服务。

- 函数原型

```
int ql_sim_deinit(void)
```

- 参数

无

- 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败，错误码详见 *ql_type.h*。

备注

若不再使用 SIM 卡服务，必须调用本函数注销 SIM 卡服务，释放资源。

2.3.3. ql_sim_get_imsi

该函数用于获取 SIM 卡的 IMSI。

- 函数原型

```
int ql_sim_get_imsi(QL_SIM_SLOT_E slot, QL_SIM_APP_TYPE_E app_type, char *imsi, int imsi_len)
```

- 参数

slot:

[In] SIM 卡卡槽编号；详见第 2.3.3.1 章。

app_type:

[In] SIM 卡应用类型；详见第 2.3.3.2 章。

imsi:

[Out] IMSI 缓存区，用于存放获取的 IMSI。

imsi_len:

[In] IMSI 缓存区长度。

- 返回值

`QL_ERR_OK` 函数执行成功
其他值 函数执行失败，错误码详见 `ql_type.h`。

2.3.3.1. QL_SIM_SLOT_E

SIM 卡卡槽编号枚举定义如下：

```
typedef enum
{
    QL_SIM_SLOT_INVALID = 0x000,
    QL_SIM_SLOT_1       = 0xB01,
    QL_SIM_SLOT_2       = 0xB02,
} QL_SIM_SLOT_E
```

- 成员

成员	描述
<code>QL_SIM_SLOT_INVALID</code>	无效卡槽
<code>QL_SIM_SLOT_1</code>	卡槽 1
<code>QL_SIM_SLOT_2</code>	卡槽 2

2.3.3.2. QL_SIM_APP_TYPE_E

SIM 卡应用类型枚举定义如下：

```
typedef enum
{
    QL_SIM_APP_TYPE_UNKNOWN = 0xB00,
    QL_SIM_APP_TYPE_3GPP    = 0xB01,
    QL_SIM_APP_TYPE_3GPP2   = 0xB02,
    QL_SIM_APP_TYPE_ISIM     = 0xB03,
} QL_SIM_APP_TYPE_E
```

- 成员

成员	描述
<code>QL_SIM_APP_TYPE_UNKNOWN</code>	未知应用类型

QL_SIM_APP_TYPE_3GPP	3GPP 类型的应用程序（SIM/USIM）
QL_SIM_APP_TYPE_3GPP2	3GPP2 类型的应用程序（RUIM/CSIM）
QL_SIM_APP_TYPE_ISIM	ISIM 应用程序

2.3.4. ql_sim_get_iccid

该函数用于获取 SIM 卡的 ICCID。

- 函数原型

```
int ql_sim_get_iccid(QL_SIM_SLOT_E slot, char *iccid, int iccid_len)
```

- 参数

slot:

[In] SIM 卡卡槽编号；详见第 2.3.3.1 章。

iccid:

[Out] ICCID 缓存区，用于存放获取的 ICCID。

iccid_len:

[In] ICCID 缓存区长度。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.5. ql_sim_get_phone_num

该函数用于获取 SIM 卡的电话号码。

- 函数原型

```
int ql_sim_get_phone_num(QL_SIM_SLOT_E slot, QL_SIM_APP_TYPE_E app_type, char *phone_num, int phone_num_len)
```

- 参数

slot:

[In] SIM 卡卡槽编号；详见第 2.3.3.1 章。

app_type:

[In] SIM 卡应用类型；详见第 2.3.3.2 章。

phone_num:

[Out] *phone_num* 缓存区，用于存放获取的号码。

phone_num:

[In] *phone_num* 缓存区长度。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.6. ql_sim_get_operators

该函数用于获取运营商列表。

- 函数原型

```
int ql_sim_get_operators(QL_SIM_SLOT_E slot, ql_sim_operator_list_t *list)
```

- 参数

slot:

[In] SIM 卡卡槽编号；详见第 2.3.3.1 章。

list:

[Out] 运营商列表；详见第 2.3.6.1 章。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.6.1. ql_sim_operator_list_t

运营商列表结构体定义如下：

```
typedef struct
{
    int len;
    ql_sim_operator_t operators[QL_SIM_NUM_OPERATOR_MAX];
} ql_sim_operator_list_t
```

● 参数

类型	参数	描述
int	<i>len</i>	运营商信息数组的数量。即 <i>operators</i> 数组的数量。
<i>ql_sim_operator_t</i>	<i>operators</i>	运营商信息。数组；有效长度由 <i>len</i> 指定，最大为 QL_SIM_NUM_OPERATOR_MAX（24）；详见第2.3.6.2章。

2.3.6.2. ql_sim_operator_t

运营商信息结构体定义如下：

```
typedef struct
{
    char mcc[QL_SIM_MCC_LENHT];
    uint8_t mnc_len;
    char mnc[QL_SIM_MNC_MAX];
} ql_sim_operator_t
```

● 参数

类型	参数	描述
char	<i>mcc</i>	移动设备国家代码。3 位。
uint8_t	<i>mnc_len</i>	移动设备国家代码长度。2 位或 3 位。
char	<i>mnc</i>	移动设备网络代码。示例如下： 00 中国移动 TD 系统 01 中国联通 GSM 系统 02 中国移动 GSM 系统 03 中国电信 CDMA 系统

2.3.7. ql_sim_enable_pin

该函数用于使能 PIN 码。

● 函数原型

```
int ql_sim_enable_pin(QL_SIM_SLOT_E slot, QL_SIM_APP_TYPE_E app_type, QL_SIM_PIN_E pin, const char *pin_value)
```

● 参数

slot:

[In] SIM 卡卡槽编号；详见第 2.3.3.1 章。

app_type:

[In] SIM 卡应用类型；详见第 2.3.3.2 章。

pin:

[In] PIN 码编号；详见第 2.3.7.1 章。

pin_value:

[In] PIN 码。字符串类型，必须以“\0”结尾，最大长度为 QL_SIM_PIN_MAX(8)。

● 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.7.1. QL_SIM_PIN_E

PIN 码编号枚举定义如下：

```
typedef enum
{
    QL_SIM_PIN_1 = 0xB01,
    QL_SIM_PIN_2 = 0xB02,
} QL_SIM_PIN_E
```

● 成员

成员	描述
QL_SIM_PIN_1	PIN1 码。由电信运行商提供，是用于 SIM 卡保密的个人识别码（Personal Identification Number）。
QL_SIM_PIN_2	PIN2 码。由供应商提供的 SIM 卡另一密码，用于限定拨号等功能的个人识别码，主要用于消除呼叫费用数据、设定通话费的计费币别和计费单位、费用限制、限定拨号。

备注

1. 启用 SIM 卡 PIN 码后，访问手机或无线通信模块需要输入 PIN 码。
2. 输入 PIN 码有次数限制，连续 3 次输入错误 PIN 码后，PIN 码会被锁定，导致 SIM 卡无法使用，需输入 PUK 码才能解锁。原始的 PIN 或 PUK 码可以通过运营商获取。

2.3.8. ql_sim_disable_pin

该函数用于禁用 PIN 码。

- 函数原型

```
int ql_sim_disable_pin(QL_SIM_SLOT_E slot, QL_SIM_APP_TYPE_E app_type,
QL_SIM_PIN_E pin, const char *pin_value)
```

- 参数

slot:

[In] SIM 卡卡槽编号；详见第 2.3.3.1 章。

app_type:

[In] SIM 卡应用类型；详见第 2.3.3.2 章。

pin:

[In] PIN 码编号；详见第 2.3.7.1 章。

pin_value:

[In] PIN 码。字符串类型，必须以“\0”结尾，最大长度为 QL_SIM_PIN_MAX(8)。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 ql_type.h。

2.3.9. ql_sim_verify_pin

该函数用于验证 PIN 码。当 SIM 卡状态为请求 PIN1/PIN2 时输入 PIN1/PIN2 码进行验证。调用该函数前需使能 PIN 锁。

- 函数原型

```
int ql_sim_verify_pin(QL_SIM_SLOT_E slot, QL_SIM_APP_TYPE_E app_type, QL_SIM_PIN_E
pin, const char *pin_value)
```

- 参数

slot:

[In] SIM 卡卡槽编号；详见第 2.3.3.1 章。

app_type:

[In] SIM 卡应用类型；详见第 2.3.3.2 章。

pin:

[In] PIN 码编号；详见第 2.3.7.1 章。

pin_value:

[In] PIN 码。字符串类型，必须以“\0”结尾，最大长度为 QL_SIM_PIN_MAX(8)。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.10. ql_sim_change_pin

该函数用于更改 PIN 码。

- 函数原型

```
int ql_sim_change_pin(QL_SIM_SLOT_E slot, QL_SIM_APP_TYPE_E app_type, QL_SIM_PIN_E pin, const char *old_pin_value, const char *new_pin_value)
```

- 参数

slot:

[In] SIM 卡卡槽编号；详见第 2.3.3.1 章。

app_type:

[In] SIM 卡应用类型；详见第 2.3.3.2 章。

pin:

[In] PIN 码编号；详见第 2.3.7.1 章。

old_pin_value:

[In] 旧的 PIN 码。字符串类型，必须以“\0”结尾，最大长度为 QL_SIM_PIN_MAX(8)。

new_pin_value:

[In] 新的 PIN 码。字符串，必须以“\0”结尾，最大长度为 QL_SIM_PIN_MAX(8)。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.11. ql_sim_unblock_pin

该函数用于解锁 SIM 卡并设置新 PIN 码。PIN 码输入错误次数超过 3 次，PIN 码会被锁定；此时需要用 PUK 码解开 SIM 卡的锁定状态。

- 函数原型

```
int ql_sim_unblock_pin(QL_SIM_SLOT_E slot, QL_SIM_APP_TYPE_E app_type,
QL_SIM_PIN_E pin, const char *puk_value, const char *pin_value)
```

- 参数

slot:

[In] SIM 卡卡槽编号；详见第 2.3.3.1 章。

app_type:

[In] SIM 卡应用类型；详见第 2.3.3.2 章。

pin:

[In] PIN 码编号；详见第 2.3.7.1 章。

puk_value:

[In] PUK 码。字符串类型，必须以“\0”结尾，最大长度为 QL_SIM_PIN_MAX(8)。

pin_value:

[In] 新的 PIN 码。字符串类型，必须以“\0”结尾，最大长度为 QL_SIM_PIN_MAX(8)。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.12. ql_sim_get_card_info

该函数用于获取 SIM 卡信息。

- 函数原型

```
int ql_sim_get_card_info(QL_SIM_SLOT_E slot, ql_sim_card_info_t *p_info)
```

- 参数

slot:

[In] SIM 卡卡槽编号；详见第 2.3.3.1 章。

p_info:

[Out] SIM 卡信息；详见第 2.3.12.1 章。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.12.1. ql_sim_card_info_t

SIM 卡信息结构体定义如下：

```
typedef struct
{
    QL_SIM_CARD_STATE_E state;
    QL_SIM_CARD_TYPE_E type;
    ql_sim_app_info_t app_3gpp;
    ql_sim_app_info_t app_3gpp2;
    ql_sim_app_info_t app_isim;
} ql_sim_card_info_t
```

- 参数

类型	参数	描述
<i>QL_SIM_CARD_STATE_E</i>	<i>state</i>	SIM 卡状态；详见第 2.3.12.2 章。
<i>QL_SIM_CARD_TYPE_E</i>	<i>type</i>	SIM 卡类型；详见第 0 章。
<i>ql_sim_app_info_t</i>	<i>app_3gpp</i>	3GPP 应用信息；详见第 2.3.12.4 章。
<i>ql_sim_app_info_t</i>	<i>app_3gpp2</i>	3GPP2 应用信息；详见第 2.3.12.4 章。
<i>ql_sim_app_info_t</i>	<i>app_isim</i>	ISIM 应用信息；详见第 2.3.12.4 章。

2.3.12.2. QL_SIM_CARD_STATE_E

SIM 卡状态类型枚举定义如下：

```
typedef enum {
    QL_SIM_CARD_STATE_UNKNOWN           = 0xB01,
    QL_SIM_CARD_STATE_ABSENT            = 0xB02,
    QL_SIM_CARD_STATE_PRESENT            = 0xB03,
    QL_SIM_CARD_STATE_ERROR_UNKNOWN      = 0xB04,
    QL_SIM_CARD_STATE_ERROR_POWER_DOWN  = 0xB05,
```

```

QL_SIM_CARD_STATE_ERROR_POLL_ERROR           = 0xB06,
QL_SIM_CARD_STATE_ERROR_NO_ATR_RECEIVED       = 0xB07,
QL_SIM_CARD_STATE_ERROR_VOLT_MISMATCH        = 0xB08,
QL_SIM_CARD_STATE_ERROR_PARITY_ERROR          = 0xB09,
QL_SIM_CARD_STATE_ERROR_SIM_TECHNICAL_PROBLEMS = 0xB0A,
} QL_SIM_CARD_STATE_E
    
```

● 成员

成员	描述
<code>QL_SIM_CARD_STATE_UNKNOWN</code>	未知状态
<code>QL_SIM_CARD_STATE_ABSENT</code>	无卡状态
<code>QL_SIM_CARD_STATE_PRESENT</code>	检测到 SIM 卡
<code>QL_SIM_CARD_STATE_ERROR_UNKNOWN</code>	未知错误
<code>QL_SIM_CARD_STATE_ERROR_POWER_DOWN</code>	SIM 卡未上电
<code>QL_SIM_CARD_STATE_ERROR_POLL_ERROR</code>	轮询错误
<code>QL_SIM_CARD_STATE_ERROR_NO_ATR_RECEIVED</code>	未收到复位应答
<code>QL_SIM_CARD_STATE_ERROR_VOLT_MISMATCH</code>	电压失配
<code>QL_SIM_CARD_STATE_ERROR_PARITY_ERROR</code>	奇偶校验错误
<code>QL_SIM_CARD_STATE_ERROR_SIM_TECHNICAL_PROBLEMS</code>	SIM 卡技术问题

2.3.12.3. QL_SIM_CARD_TYPE_E

SIM 卡类型枚举定义如下：

```

typedef enum
{
    QL_SIM_CARD_TYPE_UNKNOWN    = 0xB00,
    QL_SIM_CARD_TYPE_ICC        = 0xB01,
    QL_SIM_CARD_TYPE_UICC       = 0xB02,
} QL_SIM_CARD_TYPE_E
    
```

● 成员

成员	描述
<code>QL_SIM_CARD_TYPE_UNKNOWN</code>	未知类型

QL_SIM_CARD_TYPE_ICC	ICC 卡
QL_SIM_CARD_TYPE_UICC	UICC 卡

2.3.12.4. ql_sim_app_info_t

应用信息结构体定义如下：

```
typedef struct {
    QL_SIM_APP_STATE_E app_state;
    uint8_t pin1_num_retries;
    uint8_t puk1_num_retries;
    uint8_t pin2_num_retries;
    uint8_t puk2_num_retries;
}ql_sim_app_info_t
```

● 参数

类型	参数	描述
QL_SIM_APP_STATE_E	app_state	应用状态；详见第 2.3.12.5 章。
uint8_t	pin1_num_retries	输入 PIN1 码剩余尝试次数
uint8_t	puk1_num_retries	输入 PUK1 码剩余尝试次数
uint8_t	pin2_num_retries	输入 PIN2 码剩余尝试次数
uint8_t	puk2_num_retries	输入 PUK2 码剩余尝试次数

2.3.12.5. QL_SIM_APP_STATE_E

应用状态枚举定义如下：

```
typedef enum {
    QL_SIM_APP_STATE_UNKNOWN           = 0xB00,
    QL_SIM_APP_STATE_DETECTED          = 0xB01,
    QL_SIM_APP_STATE_PIN1_REQ          = 0xB02,
    QL_SIM_APP_STATE_PUK1_REQ          = 0xB03,
    QL_SIM_APP_STATE_INITIALIZING      = 0xB04,
    QL_SIM_APP_STATE_PERSO_CK_REQ      = 0xB05,
    QL_SIM_APP_STATE_PERSO_PUK_REQ     = 0xB06,
    QL_SIM_APP_STATE_PERSO_PERMANENTLY_BLOCKED = 0xB07,
    QL_SIM_APP_STATE_PIN1_PERM_BLOCKED = 0xB08,
```

```

    QL_SIM_APP_STATE_ILLEGAL          = 0xB09,
    QL_SIM_APP_STATE_READY            = 0xB0A,
} QL_SIM_APP_STATE_E

```

● 成员

成员	描述
<code>QL_SIM_APP_STATE_UNKNOWN</code>	未知状态
<code>QL_SIM_APP_STATE_DETECTED</code>	已检测到
<code>QL_SIM_APP_STATE_PIN1_REQ</code>	等待输入 PIN1 码
<code>QL_SIM_APP_STATE_PUK1_REQ</code>	等待输入 PUK1 码
<code>QL_SIM_APP_STATE_INITIALIZATING</code>	初始化中
<code>QL_SIM_APP_STATE_PERSO_CK_REQ</code>	等待输入 CK 密码（暂不支持）
<code>QL_SIM_APP_STATE_PERSO_PUK_REQ</code>	等待输入解锁个性化设置的密码（暂不支持）
<code>QL_SIM_APP_STATE_PERSO_PERMANENTLY_BLOCKED</code>	个性化设置被永久锁定（暂不支持）
<code>QL_SIM_APP_STATE_PIN1_PERM_BLOCKED</code>	PIN1 码被永久锁定
<code>QL_SIM_APP_STATE_ILLEGAL</code>	非法状态
<code>QL_SIM_APP_STATE_READY</code>	初始化完成

2.3.13. ql_sim_read_file

该函数用于从 SIM 卡文件读取数据。

● 函数原型

```

Int ql_sim_read_file(QL_SIM_SLOT_E slot, QL_SIM_APP_TYPE_E app_type, ql_sim_file_t
*p_file)

```

● 参数

slot:

[In] SIM 卡卡槽编号；详见第 2.3.3.1 章。

app_type:

[In] SIM 卡应用类型；详见第 2.3.3.2 章。

p_file:

[In] 文件信息；详见第 2.3.13.1 章。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.13.1. ql_sim_file_t

文件信息结构体定义如下：

```
typedef struct {
    uint32_t path_len;
    char path[QL_SIM_PATH_MAX];
    uint16_t offset;
    uint8_t record_idx;
    uint32_t data_len;
    uint8_t data[QL_SIM_DATA_MAX];
} ql_sim_file_t
```

- 参数

类型	参数	描述
uint32_t	<i>path_len</i>	文件路径长度
char	<i>path</i>	文件路径。长度由 <i>path_len</i> 指定。最大为 QL_SIM_PATH_MAX(20)。
uint16_t	<i>offset</i>	指定写入文件中数据的起始位置。仅对透明文件有效。
uint8_t	<i>record_idx</i>	记录索引。 0 读取或写入的是透明文件 大于 0 读取或写入的是循环或线性固定文件，记录的索引由该值提供
uint32_t	<i>data_len</i>	数据长度。最大为 QL_SIM_DATA_MAX。
uint8_t	<i>data</i>	待写入或待读取的数据。长度由 <i>data_len</i> 指定。

2.3.14. ql_sim_write_file

该函数用于把数据写入 SIM 卡文件。

- 函数原型

```
int ql_sim_write_file(QL_SIM_SLOT_E slot, QL_SIM_APP_TYPE_E app_type, ql_sim_file_t
```

*p_file)

- 参数

slot:

[In] SIM 卡卡槽编号；详见第 2.3.3.1 章。

app_type:

[In] SIM 卡应用类型；详见第 2.3.3.2 章。

p_file:

[In] 文件信息；详见第 2.3.13.1 章。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.15. ql_sim_get_file_info

该函数用于获取 SIM 卡文件信息。

- 函数原型

```
int ql_sim_get_file_info(QL_SIM_SLOT_E slot, QL_SIM_APP_TYPE_E app_type, ql_sim_file_info_t *p_info)
```

- 参数

slot:

[In] SIM 卡卡槽编号；详见第 2.3.3.1 章。

app_type:

[In] SIM 卡应用类型；详见第 2.3.3.2 章。

p_info:

[In/Out] 文件信息；详见第 2.3.15.1 章。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.15.1. ql_sim_file_info_t

SIM 卡文件信息结构体定义如下：

```
typedef struct {
    /* == filled by caller == */
    uint32_t path_len;
    char path[QL_SIM_PATH_MAX];
    /* == filled by callee == */
    QL_SIM_FILE_TYPE_E file_type;
    uint16_t file_size;
    uint16_t record_size;
    uint16_t record_count;
} ql_sim_file_info_t
```

● 参数

类型	参数	描述
uint32_t	<i>path_len</i>	文件路径长度。由函数调用者设置。
char	<i>path</i>	文件路径。长度由 <i>path_len</i> 指定。最大为 QL_SIM_PATH_MAX，由函数调用者设置。
QL_SIM_FILE_TYPE_E	<i>file_type</i>	SIM 卡文件类型；详见第 2.3.15.2 章。
uint16_t	<i>file_size</i>	透明文件的大小
uint16_t	<i>record_size</i>	循环或线性固定文件中每个记录的大小
uint16_t	<i>record_count</i>	循环或线性固定文件中记录个数

2.3.15.2. QL_SIM_FILE_TYPE_E

SIM 卡文件类型枚举定义如下：

```
typedef enum {
    QL_SIM_FILE_TYPE_UNKNOWN          = 0xB00,
    QL_SIM_FILE_TYPE_TRANSPARENT      = 0xB01,
    QL_SIM_FILE_TYPE_CYCLIC           = 0xB02,
    QL_SIM_FILE_TYPE_LINEAR_FIXED     = 0xB03,
} QL_SIM_FILE_TYPE_E
```

● 成员

成员	描述
<code>QL_SIM_FILE_TYPE_UNKNOWN</code>	未知类型
<code>QL_SIM_FILE_TYPE_TRANSPARENT</code>	透明文件
<code>QL_SIM_FILE_TYPE_CYCLIC</code>	循环记录文件
<code>QL_SIM_FILE_TYPE_LINEAR_FIXED</code>	线性固定文件

2.3.16. ql_sim_read_phone_book

该函数用于读取 SIM 卡电话簿中保存的联系人信息。

● 函数原型

```
int ql_sim_read_phone_book(QL_SIM_SLOT_E slot, QL_SIM_APP_TYPE_E app_type, const char *pb_path, uint8_t record_idx, ql_sim_phone_book_record_t *p_record)
```

● 参数

slot:

[In] SIM 卡卡槽编号；详见第 2.3.3.1 章。

app_type:

[In] SIM 卡应用类型；详见第 2.3.3.2 章。

pb_path:

[In] SIM 卡电话簿路径。默认值为 `QL_SIM_PB_DEFAULT_PATH`。

record_idx:

[In] 电话簿编号。

p_record:

[In/Out] 电话簿联系人信息；详见第 2.3.16.1 章。

● 返回值

`QL_ERR_OK` 函数执行成功

其他值 函数执行失败，错误码详见 `ql_type.h`。

2.3.16.1. ql_sim_phone_book_record_t

电话簿联系人信息结构体定义如下：

```
typedef struct
{
    char name[QL_SIM_PHONE_BOOK_NAME_MAX];
    char number[QL_SIM_PHONE_BOOK_NUMBER_MAX];
} ql_sim_phone_book_record_t
```

● 参数

类型	参数	描述
char	<i>name</i>	电话簿上的联系人姓名。最大长度为 QL_SIM_PHONE_BOOK_NAME_MAX(15)，由函数调用者设置。
char	<i>number</i>	电话簿上的联系人号码。最大长度为 QL_SIM_PHONE_BOOK_NUMBER_MAX(25)，由函数调用者设置。

2.3.17. ql_sim_write_phone_book

该函数用于修改电话簿内保存的联系人信息。

● 函数原型

```
int ql_sim_write_phone_book(QL_SIM_SLOT_E slot, QL_SIM_APP_TYPE_E app_type, const
char *pb_path, uint8_t record_idx, ql_sim_phone_book_record_t *p_record)
```

● 参数

slot:

[In] SIM 卡卡槽编号；详见第 2.3.3.1 章。

app_type:

[In] SIM 卡应用类型；详见第 2.3.3.2 章。

pb_path:

[In] 电话簿路径。默认值为 QL_SIM_PB_DEFAULT_PATH。

record_idx:

[In] 电话簿编号。

p_record:

[In/Out] 联系人信息；详见第 2.3.16.1 章。

- 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.18. ql_sim_open_logical_channel

该函数用于打开指定 UICC 卡的逻辑通道。

- 函数原型

```
int ql_sim_open_logical_channel(QL_SIM_SLOT_E slot, uint8_t *channel_id)
```

- 参数

slot:

[In] SIM 卡卡槽编号；详见第 2.3.3.1 章。

channel_id:

[In/Out] 逻辑通道 ID。

- 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.19. ql_sim_close_logical_channel

该函数用于关闭指定 UICC 卡的逻辑通道。

- 函数原型

```
int ql_sim_close_logical_channel(QL_SIM_SLOT_E slot, uint8_t channel_id)
```

- 参数

slot:

[In] SIM 卡卡槽编号；详见第 2.3.3.1 章。

channel_id:

[In] 逻辑通道 ID。

- 返回值

`QL_ERR_OK` 函数执行成功
其他值 函数执行失败，错误码详见 `ql_type.h`。

2.3.20. ql_sim_send_apdu

该函数用于向 SIM 卡发送 APDU 数据。调用此函数前需打开一个逻辑通道。

- 函数原型

```
int ql_sim_send_apdu(QL_SIM_SLOT_E slot, uint8_t channel_id, ql_sim_apdu_t *p_apdu)
```

- 参数

slot:

[In] SIM 卡卡槽编号；详见第 2.3.3.1 章。

channel_id:

[In] 逻辑通道 ID。

p_apdu:

[In/Out] APDU 数据；详见第 2.3.20.1 章。

- 返回值

`QL_ERR_OK` 函数执行成功
其他值 函数执行失败，错误码详见 `ql_type.h`。

2.3.20.1. ql_sim_apdu

APDU 数据结构体定义如下：

```
typedef struct
{
    uint32_t req_apdu_len;
    uint8_t req_apdu[QL_SIM_APDU_DATA_MAX];
    uint32_t resp_apdu_len;
    uint8_t resp_apdu[QL_SIM_APDU_DATA_MAX];
} ql_sim_apdu_t
```

● 参数

类型	参数	描述
uint32_t	<i>req_apdu_len</i>	请求的 APDU 数据长度。由函数调用者设置。单位：字节。
uint8_t	<i>req_apdu</i>	请求的 APDU 数据内容。长度由 <i>req_apdu_len</i> 指定，最大为 QL_SIM_APDU_DATA_MAX(1024)；由函数调用者设置。
uint32_t	<i>resp_apdu_len;</i>	响应的 APDU 数据长度
uint8_t	<i>resp_apdu</i>	响应的 APDU 数据。长度由 <i>req_apdu_len</i> 指定，最大为 QL_SIM_APDU_DATA_MAX(1024)。

2.3.21. ql_sim_set_card_status_cb

该函数用于设置 SIM 卡状态接收回调函数。

● 函数原型

```
int ql_sim_set_card_status_cb(ql_sim_card_status_cb_f cb)
```

● 参数

cb:

[In] SIM 卡状态接收回调函数；详见第 2.3.21.1 章。

● 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.21.1. ql_sim_card_status_cb_f

该回调函数接收 SIM 卡状态信息。

● 函数原型

```
typedef void (*ql_sim_card_status_cb_f)(QL_SIM_SLOT_E slot, ql_sim_card_info_t *p_info)
```

● 参数

slot:

[In] SIM 卡卡槽编号；详见第 2.3.3.1 章。

p_info:

[In] SIM 卡信息；详见第 2.3.12.1 章。

- 返回值

无

2.3.22. ql_sim_set_service_error_cb

该函数用于设置 SIM 卡服务异常接收回调函数。

- 函数原型

```
int ql_sim_set_service_error_cb(ql_sim_service_error_cb_f cb)
```

- 参数

cb:

[In] SIM 卡服务异常接收回调函数；详见第 2.3.22.1 章。

- 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败，错误码详见 *ql_type.h*。

2.3.22.1. ql_sim_service_error_cb_f

该回调函数接收 SIM 卡服务异常信息。

- 函数原型

```
typedef void (*ql_sim_service_error_cb_f)(int error)
```

- 参数

error:

[In] 服务错误码。当前只有在服务端异常退出时才触发回调函数，错误码为 **QL_ERR_ABORTED**。

- 返回值

无

3 示例

本章所述代码示例均摘自 `sample\test_sdk_api\m_sim.c`，用户可自行查看接口函数的完整示例。

备注

1. 程序启动后，必须调用 `ql_sim_init()` 初始化 SIM 卡服务。
2. 程序退出前或不再使用 SIM 卡服务时，必须调用 `ql_sim_deinit()` 释放资源。

3.1. 获取 SIM 卡的 IMSI

```
static void item_ql_sim_get_imsi(void)
{
    int ret = 0;
    char imsi[QL_SIM_IMSI_LENGTH+1] = {0};
    int input = 0;
    QL_SIM_SLOT_E slot;
    QL_SIM_APP_TYPE_E app_type;

    printf("test ql_sim_get_imsi: \n");
    printf("please enter slot(1 or 2): ");
    scanf("%d", &input);
    getchar();
    if (1 == input)
    {
        slot = QL_SIM_SLOT_1;
    }
    else if (2 == input)
    {
        slot = QL_SIM_SLOT_2;
    }
    else
    {
        printf("bad slot: %d\n", input);
        return;
    }
}
```

```

}

printf("please enter app type(0-unknown, 1-3gpp, 2-3gpp2, 3-isim): ");
scanf("%d", &input);
getchar();
switch (input)
{
    case 0:
        app_type = QL_SIM_APP_TYPE_UNKNOWN;
        break;
    case 1:
        app_type = QL_SIM_APP_TYPE_3GPP;
        break;
    case 2:
        app_type = QL_SIM_APP_TYPE_3GPP2;
        break;
    case 3:
        app_type = QL_SIM_APP_TYPE_ISIM;
        break;
    default:
        printf("bad app type: %d\n", input);
        return;
}

ret = ql_sim_get_imsi(slot, app_type, imsi, sizeof(imsi));
if (ret == QL_ERR_OK)
{
    printf("IMSI: %s\n", imsi);
}
else
{
    printf("failed, ret = %d\n", ret);
}
}

```

3.2. 获取 SIM 卡的 ICCID

```

static void item_ql_sim_get_iccid(void)
{
    int ret = 0;
    char iccid[QL_SIM_ICCID_LENGTH+1] = {0};
}

```

```
int input = 0;
QL_SIM_SLOT_E slot;

printf("test ql_sim_get_iccid: \n");
printf("please enter slot(1 or 2): ");
scanf("%d", &input);
getchar();
if (1 == input)
{
    slot = QL_SIM_SLOT_1;
}
else if (2 == input)
{
    slot = QL_SIM_SLOT_2;
}
else
{
    printf("bad slot: %d\n", input);
    return;
}

ret = ql_sim_get_iccid(slot, iccid, sizeof(iccid));
if (ret == QL_ERR_OK)
{
    printf("ICCID: %s\n", iccid);
}
else
{
    printf("failed, ret = %d\n", ret);
}
}
```

3.3. 获取 SIM 卡的电话号码

```
static void item_ql_sim_get_phone_num(void)
{
    int ret = 0;
    char num[QL_SIM_PHONE_NUMBER_MAX+1] = {0};
    int input = 0;
    QL_SIM_SLOT_E slot;
    QL_SIM_APP_TYPE_E app_type;
```

```

printf("test ql_sim_get_phone_num: \n");
printf("please enter slot(1 or 2): ");
scanf("%d", &input);
getchar();
if (1 == input)
{
    slot = QL_SIM_SLOT_1;
}
else if (2 == input)
{
    slot = QL_SIM_SLOT_2;
}
else
{
    printf("bad slot: %d\n", input);
    return;
}

printf("please enter app type(0-unknown, 1-3gpp, 2-3gpp2, 3-isim): ");
scanf("%d", &input);
getchar();
switch (input)
{
    case 0:
        app_type = QL_SIM_APP_TYPE_UNKNOWN;
        break;
    case 1:
        app_type = QL_SIM_APP_TYPE_3GPP;
        break;
    case 2:
        app_type = QL_SIM_APP_TYPE_3GPP2;
        break;
    case 3:
        app_type = QL_SIM_APP_TYPE_ISIM;
        break;
    default:
        printf("bad app type: %d\n", input);
        return;
}

ret = ql_sim_get_phone_num(slot, app_type, num, sizeof(num));
if (ret == QL_ERR_OK)
{

```

```

        printf("Phone number: %s\n", num);
    }
    else
    {
        printf("failed, ret = %d\n", ret);
    }
}

```

3.4. 获取运营商列表

```

static void item_ql_sim_get_operators(void)
{
    int ret = 0;
    ql_sim_operator_list_t list = {0};
    int input = 0;
    QL_SIM_SLOT_E slot;

    printf("test ql_sim_get_operators: \n");
    printf("please enter slot(1 or 2): ");
    scanf("%d", &input);
    getchar();
    if (1 == input)
    {
        slot = QL_SIM_SLOT_1;
    }
    else if (2 == input)
    {
        slot = QL_SIM_SLOT_2;
    }
    else
    {
        printf("bad slot: %d\n", input);
        return;
    }

    ret = ql_sim_get_operators(slot, &list);
    if (ret == QL_ERR_OK)
    {
        if (0 == list.len)
        {
            printf("No operators found\n");
        }
    }
}

```

```

    }
    else
    {
        int i = 0;
        printf("found %d opertators:\n", list.len);
        for (i = 0; i < list.len; i++)
        {
            printf(" #02d: ", i + 1);
            print_ascii("MCC: ", "", list.operators[i].mcc, (int)sizeof(list.operators[i].mcc));
            print_ascii(", MNC: ", "\n", list.operators[i].mnc, list.operators[i].mnc_len);
        }
    }
}
else
{
    printf("failed, ret = %d\n", ret);
}
}

```

3.5. PIN 码操作

3.5.1. 使能 PIN 码

```

static void item_ql_sim_enable_pin(void)
{
    int ret = 0;
    int len = 0;
    char c;
    char pin_value[QL_SIM_PIN_MAX*2] = {0};
    int input = 0;
    QL_SIM_SLOT_E slot;
    QL_SIM_APP_TYPE_E app_type;
    QL_SIM_PIN_E pin;

    printf("test ql_sim_enable_pin: \n");
    printf("please enter slot(1 or 2): ");
    scanf("%d", &input);
    getchar();
    if (1 == input)
    {
        slot = QL_SIM_SLOT_1;
    }
}

```

```

else if (2 == input)
{
    slot = QL_SIM_SLOT_2;
}
else
{
    printf("bad slot: %d\n", input);
    return;
}

printf("please enter app type(0-unknown, 1-3gpp, 2-3gpp2, 3-isim): ");
scanf("%d", &input);
getchar();
switch (input)
{
    case 0:
        app_type = QL_SIM_APP_TYPE_UNKNOWN;
        break;
    case 1:
        app_type = QL_SIM_APP_TYPE_3GPP;
        break;
    case 2:
        app_type = QL_SIM_APP_TYPE_3GPP2;
        break;
    case 3:
        app_type = QL_SIM_APP_TYPE_ISIM;
        break;
    default:
        printf("bad app type: %d\n", input);
        return;
}

printf("please enter pin(1 or 2): ");
scanf("%d", &input);
getchar();
if (1 == input)
{
    pin = QL_SIM_PIN_1;
}
else if (2 == input)
{
    pin = QL_SIM_PIN_2;
}
else

```

```

{
    printf("bad pin: %d\n", input);
    return;
}

printf("please enter pin value(at most %d digit): ", QL_SIM_PIN_MAX);
if (NULL == fgets(pin_value, sizeof(pin_value), stdin))
{
    printf("can not read pin value\n");
    return;
}
len = strlen(pin_value);
if ('\n' == pin_value[len-1])
{
    pin_value[len-1] = 0;
    len--;
}
printf("pin value: %s\n", pin_value);

printf("proceed? [y/n]: ");
c = getchar();
if ('\n' != c)
{
    getchar();
}
if ('Y' != c && 'y' != c)
{
    printf("abort\n");
    return;
}

ret = ql_sim_enable_pin(slot, app_type, pin, pin_value);
if (ret == QL_ERR_OK)
{
    printf("ok\n");
}
else
{
    printf("failed, ret = %d\n", ret);
}
}

```

3.5.2. 禁用 PIN 码

代码示例与第 3.5.1 章类似，区别仅为调用的函数替换为 `ql_sim_disable_pin()`。

3.5.3. 验证 PIN 码

代码示例与第 3.5.1 章类似，区别仅为调用的函数替换为 `ql_sim_verify_pin()`。

3.5.4. 更改 PIN 码

```
static void item_ql_sim_change_pin(void)
{
    int ret = 0;
    int old_len = 0;
    int new_len = 0;
    char c;
    char old_pin_value[QL_SIM_PIN_MAX*2] = {0};
    char new_pin_value[QL_SIM_PIN_MAX*2] = {0};
    int input = 0;
    QL_SIM_SLOT_E slot;
    QL_SIM_APP_TYPE_E app_type;
    QL_SIM_PIN_E pin;

    printf("test ql_sim_change_pin: \n");
    printf("please enter slot(1 or 2): ");
    scanf("%d", &input);
    getchar();
    if (1 == input)
    {
        slot = QL_SIM_SLOT_1;
    }
    else if (2 == input)
    {
        slot = QL_SIM_SLOT_2;
    }
    else
    {
        printf("bad slot: %d\n", input);
        return;
    }

    printf("please enter app type(0-unknown, 1-3gpp, 2-3gpp2, 3-isim): ");
    scanf("%d", &input);
```

```

getchar();
switch (input)
{
    case 0:
        app_type = QL_SIM_APP_TYPE_UNKNOWN;
        break;
    case 1:
        app_type = QL_SIM_APP_TYPE_3GPP;
        break;
    case 2:
        app_type = QL_SIM_APP_TYPE_3GPP2;
        break;
    case 3:
        app_type = QL_SIM_APP_TYPE_ISIM;
        break;
    default:
        printf("bad app type: %d\n", input);
        return;
}

printf("please enter pin(1 or 2): ");
scanf("%d", &input);
getchar();
if (1 == input)
{
    pin = QL_SIM_PIN_1;
}
else if (2 == input)
{
    pin = QL_SIM_PIN_2;
}
else
{
    printf("bad pin: %d\n", input);
    return;
}

printf("please enter old pin value(at most %d digit): ", QL_SIM_PIN_MAX);
if (NULL == fgets(old_pin_value, sizeof(old_pin_value), stdin))
{
    printf("can not read old pin value\n");
    return;
}
old_len = strlen(old_pin_value);

```

```

if ('\n' == old_pin_value[old_len-1])
{
    old_pin_value[old_len-1] = 0;
    old_len--;
}

printf("please enter new pin value(at most %d digit): ", QL_SIM_PIN_MAX);
if (NULL == fgets(new_pin_value, sizeof(new_pin_value), stdin))
{
    printf("can not read new pin value\n");
    return;
}
new_len = strlen(new_pin_value);
if ('\n' == new_pin_value[new_len-1])
{
    new_pin_value[new_len-1] = 0;
    new_len--;
}
printf("old pin value: %s\n", old_pin_value);
printf("new pin value: %s\n", new_pin_value);

printf("proceed? [y/n]: ");
c = getchar();
if ('\n' != c)
{
    getchar();
}
if ('Y' != c && 'y' != c)
{
    printf("abort\n");
    return;
}

ret = ql_sim_change_pin(slot, app_type, pin, old_pin_value, new_pin_value);
if (ret == QL_ERR_OK)
{
    printf("ok\n");
}
else
{
    printf("failed, ret = %d\n", ret);
}
}

```

3.5.5. 解锁 SIM 卡并设置新 PIN 码

```
static void item_ql_sim_unblock_pin(void)
{
    int ret = 0;
    int len = 0;
    int new_len = 0;
    char c;
    char puk_value[QL_SIM_PIN_MAX*2] = {0};
    char new_pin_value[QL_SIM_PIN_MAX*2] = {0};
    int input = 0;
    QL_SIM_SLOT_E slot;
    QL_SIM_APP_TYPE_E app_type;
    QL_SIM_PIN_E pin;

    printf("test ql_sim_unblock_pin: \n");
    printf("please enter slot(1 or 2): ");
    scanf("%d", &input);
    getchar();
    if (1 == input)
    {
        slot = QL_SIM_SLOT_1;
    }
    else if (2 == input)
    {
        slot = QL_SIM_SLOT_2;
    }
    else
    {
        printf("bad slot: %d\n", input);
        return;
    }

    printf("please enter app type(0-unknown, 1-3gpp, 2-3gpp2, 3-isim): ");
    scanf("%d", &input);
    getchar();
    switch (input)
    {
        case 0:
            app_type = QL_SIM_APP_TYPE_UNKNOWN;
            break;
        case 1:
            app_type = QL_SIM_APP_TYPE_3GPP;
            break;
```

```

        case 2:
            app_type = QL_SIM_APP_TYPE_3GPP2;
            break;
        case 3:
            app_type = QL_SIM_APP_TYPE_ISIM;
            break;
        default:
            printf("bad app type: %d\n", input);
            return;
    }

    printf("please enter pin(1 or 2): ");
    scanf("%d", &input);
    getchar();
    if (1 == input)
    {
        pin = QL_SIM_PIN_1;
    }
    else if (2 == input)
    {
        pin = QL_SIM_PIN_2;
    }
    else
    {
        printf("bad pin: %d\n", input);
        return;
    }

    printf("please enter puk value(at most %d digit): ", QL_SIM_PIN_MAX);
    if (NULL == fgets(puk_value, sizeof(puk_value), stdin))
    {
        printf("can not read old pin value\n");
        return;
    }
    len = strlen(puk_value);
    if ('\n' == puk_value[len-1])
    {
        puk_value[len-1] = 0;
        len--;
    }

    printf("please enter new pin value(at most %d digit): ", QL_SIM_PIN_MAX);
    if (NULL == fgets(new_pin_value, sizeof(new_pin_value), stdin))
    {

```

```

        printf("can not read new pin value\n");
        return;
    }
    new_len = strlen(new_pin_value);
    if ('\n' == new_pin_value[new_len-1])
    {
        new_pin_value[new_len-1] = 0;
        new_len--;
    }
    printf("    puk value: %s\n", puk_value);
    printf("new pin value: %s\n", new_pin_value);

    printf("proceed? [y/n]: ");
    c = getchar();
    if ('\n' != c)
    {
        getchar();
    }
    if ('Y' != c && 'y' != c)
    {
        printf("abort\n");
        return;
    }

    ret = ql_sim_unblock_pin(slot, app_type, pin, puk_value, new_pin_value);
    if (ret == QL_ERR_OK)
    {
        printf("ok\n");
    }
    else
    {
        printf("failed, ret = %d\n", ret);
    }
}

```

3.6. SIM 卡文件操作

3.6.1. 从 SIM 卡文件读取数据

```

static void item_ql_sim_read_file(void)
{
    int ret = 0;

```

```

int input = 0;
int len = 0;
QL_SIM_SLOT_E slot;
QL_SIM_APP_TYPE_E app_type;
ql_sim_file_t file = {0};

printf("test ql_sim_read_file: \n");
printf("please enter slot(1 or 2): ");
scanf("%d", &input);
getchar();
if (1 == input)
{
    slot = QL_SIM_SLOT_1;
}
else if (2 == input)
{
    slot = QL_SIM_SLOT_2;
}
else
{
    printf("bad slot: %d\n", input);
    return;
}

printf("please enter app type(0-unknown, 1-3gpp, 2-3gpp2, 3-isim): ");
scanf("%d", &input);
getchar();
switch (input)
{
    case 0:
        app_type = QL_SIM_APP_TYPE_UNKNOWN;
        break;
    case 1:
        app_type = QL_SIM_APP_TYPE_3GPP;
        break;
    case 2:
        app_type = QL_SIM_APP_TYPE_3GPP2;
        break;
    case 3:
        app_type = QL_SIM_APP_TYPE_ISIM;
        break;
    default:
        printf("bad app type: %d\n", input);
        return;
}

```

```

}

printf("please enter file path(at most %d hex[0 • 9A-F], e.g 3F002FE2): ", QL_SIM_PATH_MAX);
if (NULL == fgets(file.path, QL_SIM_PATH_MAX, stdin))
{
    printf("can not read file path\n");
    return;
}
len = strlen(file.path);
if ('\n' == file.path[len-1])
{
    file.path[len-1] = 0;
    len--;
}
file.path_len = (uint32_t)len;

printf("please enter record index(0 for transparent access): ");
scanf("%hhu", (uint8_t *)&file.record_idx);
getchar();

ret = ql_sim_read_file(slot, app_type, &file);
if (ret == QL_ERR_OK)
{
    printf("data length: %u\n", file.data_len);
    uint32_t i = 0;
    printf("data: ");
    for (i = 0; i < file.data_len; i++)
    {
        printf("%02x ", file.data[i]);
    }
    printf("\n");
}
else
{
    printf("failed, ret = %d\n", ret);
}
}

```

3.6.2. 把数据写入 SIM 卡文件

```
static void item_ql_sim_write_file(void)
{
    int ret = 0;
    int input = 0;
    int len = 0;
    uint8_t v;
    QL_SIM_SLOT_E slot;
    QL_SIM_APP_TYPE_E app_type;
    ql_sim_file_t file = {0};

    printf("test ql_sim_write_file: \n");
    printf("please enter slot(1 or 2): ");
    scanf("%d", &input);
    getchar();
    if (1 == input)
    {
        slot = QL_SIM_SLOT_1;
    }
    else if (2 == input)
    {
        slot = QL_SIM_SLOT_2;
    }
    else
    {
        printf("bad slot: %d\n", input);
        return;
    }

    printf("please enter app type(0-unknown, 1-3gpp, 2-3gpp2, 3-isim): ");
    scanf("%d", &input);
    getchar();
    switch (input)
    {
        case 0:
            app_type = QL_SIM_APP_TYPE_UNKNOWN;
            break;
        case 1:
            app_type = QL_SIM_APP_TYPE_3GPP;
            break;
        case 2:
            app_type = QL_SIM_APP_TYPE_3GPP2;
            break;
    }
}
```

```

        case 3:
            app_type = QL_SIM_APP_TYPE_ISIM;
            break;
        default:
            printf("bad app type: %d\n", input);
            return;
    }

    printf("please enter file path(at most %d hex[0 • 9A-F], e.g 3F002FE2): ", QL_SIM_PATH_MAX);
    if (NULL == fgets(file.path, QL_SIM_PATH_MAX, stdin))
    {
        printf("can not read file path\n");
        return;
    }
    len = strlen(file.path);
    if ('\n' == file.path[len-1])
    {
        file.path[len-1] = 0;
        len--;
    }
    file.path_len = (uint32_t)len;

    printf("please enter record index(0 for transparent access): ");
    scanf("%hhu", (uint8_t *)&file.record_idx);
    getchar();

    printf("please enter data(hex, end with `q`): ");
    while (1 == scanf("%hhx", &v))
    {
        file.data[file.data_len++] = v;
    }
    getchar(); // read `q`
    getchar(); // read '\n'

    printf("please enter data offset: ");
    scanf("%hu", &file.offset);
    getchar();

    ret = ql_sim_write_file(slot, app_type, &file);
    if (ret == QL_ERR_OK)
    {
        printf("ok\n");
    }
    else

```

```

{
    printf("failed, ret = %d\n", ret);
}
}

```

3.6.3. 获取 SIM 卡文件信息

```

static void item_ql_sim_get_file_info(void)
{
    int ret = 0;
    int input = 0;
    int len = 0;
    QL_SIM_SLOT_E slot;
    QL_SIM_APP_TYPE_E app_type;
    ql_sim_file_info_t info = {0};

    printf("test ql_sim_get_file_info: \n");
    printf("please enter slot(1 or 2): ");
    scanf("%d", &input);
    getchar();
    if (1 == input)
    {
        slot = QL_SIM_SLOT_1;
    }
    else if (2 == input)
    {
        slot = QL_SIM_SLOT_2;
    }
    else
    {
        printf("bad slot: %d\n", input);
        return;
    }

    printf("please enter app type(0-unknown, 1-3gpp, 2-3gpp2, 3-isim): ");
    scanf("%d", &input);
    getchar();
    switch (input)
    {
        case 0:
            app_type = QL_SIM_APP_TYPE_UNKNOWN;
            break;
        case 1:

```

```

        app_type = QL_SIM_APP_TYPE_3GPP;
        break;
    case 2:
        app_type = QL_SIM_APP_TYPE_3GPP2;
        break;
    case 3:
        app_type = QL_SIM_APP_TYPE_ISIM;
        break;
    default:
        printf("bad app type: %d\n", input);
        return;
}

printf("please enter file path(at most %d hex[0 • 9A-F], e.g 3F002FE2): ", QL_SIM_PATH_MAX);
if (NULL == fgets(info.path, QL_SIM_PATH_MAX, stdin))
{
    printf("can not read file path\n");
    return;
}
len = strlen(info.path);
if ('\n' == info.path[len-1])
{
    info.path[len-1] = 0;
    len--;
}
info.path_len = (uint32_t)len;

ret = ql_sim_get_file_info(slot, app_type, &info);
if (ret == QL_ERR_OK)
{
    printf("===== FILE INFO =====\n");
    printf("path: %s\n", info.path);
    printf("type: %s\n", file_type_desc(info.file_type));
    printf("file size: %hu\n", info.file_size);
    printf("record size: %hu\n", info.record_size);
    printf("record count: %hu\n", info.record_count);
}
else
{
    printf("failed, ret = %d\n", ret);
}
}

```

以文件 3F002FE2 为例，执行结果如下：

```

please enter slot(1 or 2): 1
please enter app type(0-unknown, 1-3gpp, 2-3gpp2, 3-isim): 1
please enter file path(at most 20 hex[0-9A-F], e.g 3F002FE2): 3F002FE2
===== FILE INFO =====
path: 3F002FE2
type: transparent
file size: 10
record size: 0
record count: 0

```

3.7. 获取 SIM 卡状态

```

static void item_ql_sim_get_card_info(void)
{
    int ret = 0;
    ql_sim_card_info_t info = {0};
    int input = 0;
    QL_SIM_SLOT_E slot;

    printf("test ql_sim_get_card_info: \n");
    printf("please enter slot(1 or 2): ");
    scanf("%d", &input);
    getchar();
    if (1 == input)
    {
        slot = QL_SIM_SLOT_1;
    }
    else if (2 == input)
    {
        slot = QL_SIM_SLOT_2;
    }
    else
    {
        printf("bad slot: %d\n", input);
        return;
    }

    ret = ql_sim_get_card_info(slot, &info);
    if (ret != QL_ERR_OK)
    {
        printf("failed, ret = %d\n", ret);
        return;
    }
}

```

```

}
printf("===== CARD INFO =====\n");
printf("state: %s\n", card_state_desc(info.state));
printf("type: %s\n", card_type_desc(info.type));
printf("3gpp:\n");
printf(" app state: %s\n", card_app_state_desc(info.app_3gpp.app_state));
printf(" PIN 1 retries: %hu\n", info.app_3gpp.pin1_num_retries);
printf(" PUK 1 retries: %hu\n", info.app_3gpp.puk1_num_retries);
printf(" PIN 2 retries: %hu\n", info.app_3gpp.pin2_num_retries);
printf(" PUK 2 retries: %hu\n", info.app_3gpp.puk2_num_retries);
printf("3gpp2:\n");
printf(" app state: %s\n", card_app_state_desc(info.app_3gpp2.app_state));
printf(" PIN 1 retries: %hu\n", info.app_3gpp2.pin1_num_retries);
printf(" PUK 1 retries: %hu\n", info.app_3gpp2.puk1_num_retries);
printf(" PIN 2 retries: %hu\n", info.app_3gpp2.pin2_num_retries);
printf(" PUK 2 retries: %hu\n", info.app_3gpp2.puk2_num_retries);
printf("isim:\n");
printf(" app state: %s\n", card_app_state_desc(info.app_isim.app_state));
printf(" PIN 1 retries: %hu\n", info.app_isim.pin1_num_retries);
printf(" PUK 1 retries: %hu\n", info.app_isim.puk1_num_retries);
printf(" PIN 2 retries: %hu\n", info.app_isim.pin2_num_retries);
printf(" PUK 2 retries: %hu\n", info.app_isim.puk2_num_retries);
}

```

执行结果如下：

```

===== CARD INFO =====
state: present
type: UICC
3gpp:
 app state: ready
 PIN 1 retries: 3
 PUK 1 retries: 10
 PIN 2 retries: 1
 PUK 2 retries: 10
3gpp2:
 app state: N/A
 PIN 1 retries: 0
 PUK 1 retries: 0
 PIN 2 retries: 0
 PUK 2 retries: 0
isim:
 app state: N/A
 PIN 1 retries: 0
 PUK 1 retries: 0
 PIN 2 retries: 0
 PUK 2 retries: 0

```

3.8. 电话簿操作

3.8.1. 读取电话簿信息

```
static void item_ql_sim_read_phone_book(void)
{
    int ret = 0;
    int input = 0;
    QL_SIM_SLOT_E slot;
    QL_SIM_APP_TYPE_E app_type;
    uint8_t record_idx = 0;
    ql_sim_phone_book_record_t record = {0};

    printf("test ql_sim_read_phone_book: \n");
    printf("please enter slot(1 or 2): ");
    scanf("%d", &input);
    getchar();
    if (1 == input)
    {
        slot = QL_SIM_SLOT_1;
    }
    else if (2 == input)
    {
        slot = QL_SIM_SLOT_2;
    }
    else
    {
        printf("bad slot: %d\n", input);
        return;
    }

    printf("please enter app type(0-unknown, 1-3gpp, 2-3gpp2, 3-isim): ");
    scanf("%d", &input);
    getchar();
    switch (input)
    {
        case 0:
            app_type = QL_SIM_APP_TYPE_UNKNOWN;
            break;
        case 1:
            app_type = QL_SIM_APP_TYPE_3GPP;
            break;
        case 2:
```

```

        app_type = QL_SIM_APP_TYPE_3GPP2;
        break;
    case 3:
        app_type = QL_SIM_APP_TYPE_ISIM;
        break;
    default:
        printf("bad app type: %d\n", input);
        return;
}

printf("please enter record index: ");
scanf("%hhu", &record_idx);
getchar();

ret = ql_sim_read_phone_book(slot, app_type, QL_SIM_PB_DEFAULT_PATH, record_idx,
&record);
if (ret == QL_ERR_OK)
{
    printf("Name: %s\n", record.name);
    printf("Number: %s\n", record.number);
}
else
{
    printf("failed, ret = %d\n", ret);
}
}

```

3.8.2. 修改电话簿信息

```

static void item_ql_sim_write_phone_book(void)
{4
    int ret = 0;
    int input = 0;
    int len = 0;
    QL_SIM_SLOT_E slot;
    QL_SIM_APP_TYPE_E app_type;
    uint8_t record_idx = 0;
    ql_sim_phone_book_record_t record = {0};

    printf("test ql_sim_write_phone_book: \n");
    printf("please enter slot(1 or 2): ");
    scanf("%d", &input);

```

```

getchar();
if (1 == input)
{
    slot = QL_SIM_SLOT_1;
}
else if (2 == input)
{
    slot = QL_SIM_SLOT_2;
}
else
{
    printf("bad slot: %d\n", input);
    return;
}

printf("please enter app type(0-unknown, 1-3gpp, 2-3gpp2, 3-isim): ");
scanf("%d", &input);
getchar();
switch (input)
{
    case 0:
        app_type = QL_SIM_APP_TYPE_UNKNOWN;
        break;
    case 1:
        app_type = QL_SIM_APP_TYPE_3GPP;
        break;
    case 2:
        app_type = QL_SIM_APP_TYPE_3GPP2;
        break;
    case 3:
        app_type = QL_SIM_APP_TYPE_ISIM;
        break;
    default:
        printf("bad app type: %d\n", input);
        return;
}

printf("please enter record index: ");
scanf("%hhu", &record_idx);
getchar();

printf("please enter name(at most %d chars): ", QL_SIM_PHONE_BOOK_NAME_MAX - 1);
if (NULL == fgets(record.name, QL_SIM_PHONE_BOOK_NAME_MAX, stdin))
{

```

```

        printf("\nname will be set to 0\n");
    }
    else
    {
        len = strlen(record.name);
        if ('\n' == record.name[len-1])
        {
            record.name[len-1] = 0;
        }
    }

    printf("please enter number(at most %d digits): ", QL_SIM_PHONE_BOOK_NUMBER_MAX - 1);
    if (NULL == fgets(record.number, QL_SIM_PHONE_BOOK_NUMBER_MAX, stdin))
    {
        printf("\nnumber will be set to 0\n");
    }
    else
    {
        len = strlen(record.number);
        if ('\n' == record.number[len-1])
        {
            record.number[len-1] = 0;
        }
    }

    ret = ql_sim_write_phone_book(slot, app_type, QL_SIM_PB_DEFAULT_PATH, record_idx,
    &record);
    if (ret == QL_ERR_OK)
    {
        printf("ok\n");
    }
    else
    {
        printf("failed, ret = %d\n", ret);
    }
}

```

3.9. 发送 APDU

3.9.1. 打开逻辑通道

```
static void item_ql_sim_open_logical_channel(void)
{
    int ret = 0;
    int input = 0;
    QL_SIM_SLOT_E slot;
    uint8_t channel_id = 0;

    printf("test ql_sim_open_logical_channel: \n");
    printf("please enter slot(1 or 2): ");
    scanf("%d", &input);
    getchar();
    if (1 == input)
    {
        slot = QL_SIM_SLOT_1;
    }
    else if (2 == input)
    {
        slot = QL_SIM_SLOT_2;
    }
    else
    {
        printf("bad slot: %d\n", input);
        return;
    }

    ret = ql_sim_open_logical_channel(slot, &channel_id);
    if (ret == QL_ERR_OK)
    {
        printf("channel id: %hhu\n", channel_id);
    }
    else
    {
        printf("failed, ret = %d\n", ret);
    }
}
```

3.9.2. 关闭逻辑通道

```
static void item_ql_sim_close_logical_channel(void)
{
    int ret = 0;
    int input = 0;
    QL_SIM_SLOT_E slot;
    uint8_t channel_id = 0;

    printf("test ql_sim_close_logical_channel: \n");
    printf("please enter slot(1 or 2): ");
    scanf("%d", &input);
    getchar();
    if (1 == input)
    {
        slot = QL_SIM_SLOT_1;
    }
    else if (2 == input)
    {
        slot = QL_SIM_SLOT_2;
    }
    else
    {
        printf("bad slot: %d\n", input);
        return;
    }

    printf("please enter channel id: ");
    scanf("%hhu", &channel_id);
    getchar();

    ret = ql_sim_close_logical_channel(slot, channel_id);
    if (ret == QL_ERR_OK)
    {
        printf("ok\n");
    }
    else
    {
        printf("failed, ret = %d\n", ret);
    }
}
```

3.9.3. 发送 APDU

```
static void item_ql_sim_send_apdu(void)
{
    int ret = 0;
    int input = 0;
    uint8_t v = 0;
    QL_SIM_SLOT_E slot;
    ql_sim_apdu_t *p_apdu = NULL;
    uint8_t channel_id = 0;

    printf("test ql_sim_send_apdu: \n");

    p_apdu = calloc(1, sizeof(*p_apdu));
    if (NULL == p_apdu)
    {
        printf("run out of memory\n");
        return;
    }

    printf("please enter slot(1 or 2): ");
    scanf("%d", &input);
    getchar();
    if (1 == input)
    {
        slot = QL_SIM_SLOT_1;
    }
    else if (2 == input)
    {
        slot = QL_SIM_SLOT_2;
    }
    else
    {
        printf("bad slot: %d\n", input);
        free(p_apdu);
        return;
    }

    printf("please enter channel id: ");
    scanf("%hhu", &channel_id);
    getchar();

    printf("please enter apdu data(hex, end with `q'): ");
    while (1 == scanf("%hhx", &v))
```

```

{
    p_apdu->req_apdu[p_apdu->req_apdu_len++] = v;
}
getchar(); // read `q'
getchar(); // read '\n'

ret = ql_sim_send_apdu(slot, channel_id, p_apdu);
if (ret == QL_ERR_OK)
{
    uint32_t i = 0;
    printf("response apdu: ");
    for (i = 0; i < p_apdu->resp_apdu_len; i++)
    {
        printf("%c", p_apdu->resp_apdu[i]);
    }
    printf("\n");
}
else
{
    printf("failed, ret = %d\n", ret);
}
free(p_apdu);
}

```

4 附录 参考文档及术语缩写

表 2：参考文档

文档名称
[1] Quectel_EC200A-CN(TA)_QuecOpen_快速开发指导

表 3：术语缩写

缩写	英文全称	中文全称
3GPP	3rd Generation Partnership Project	第三代合作伙伴计划
3GPP2	3rd Generation Partnership Project 2	第三代合作伙伴计划 2
APDU	Application Protocol Data Unit	应用协议数据单元
API	Application Programming Interface	应用程序编程接口
ASCII	American Standard Code for Information Interchange	美国信息交换标准码
CK	Control Key	控制密码
CSIM	CDMA2000 Subscriber Identity Module	CDMA2000 用户识别模块
GID1	Group Identifier Level 1	组识别符（级别 1）
GID2	Group Identifier Level 2	组识别符（级别 2）
GSM	Global System for Mobile	全球移动通信系统
ICCID	Integrated Circuit Card ID	集成电路卡识别码
ID	Identifier	标识符
IMSI	International Mobile Subscriber Identity	国际移动用户识别码
IoT	Internet of Things	物联网

ISIM	IP Multimedia Service Identity Module	IP 多媒体服务身份模块
MCC	Mobile Country Code	移动设备国家代码
MNC	Mobile Network Code	移动设备网络代码
PIN	Personal Identification Number	个人识别密码
PUK	PIN Unblocking Key	PIN 码解锁密码
RIM	Remote Interference Management	远程干扰管理
RUIM	Removable User Identity Module	可移动用户识别模块
SDK	Software Development Kit	软件开发工具包
SIM	Subscriber Identification Module	用户识别模块
UICC	Universal Integrated Circuit Card	通用集成电路卡
USIM	Universal Subscriber Identity Module	全球用户识别模块